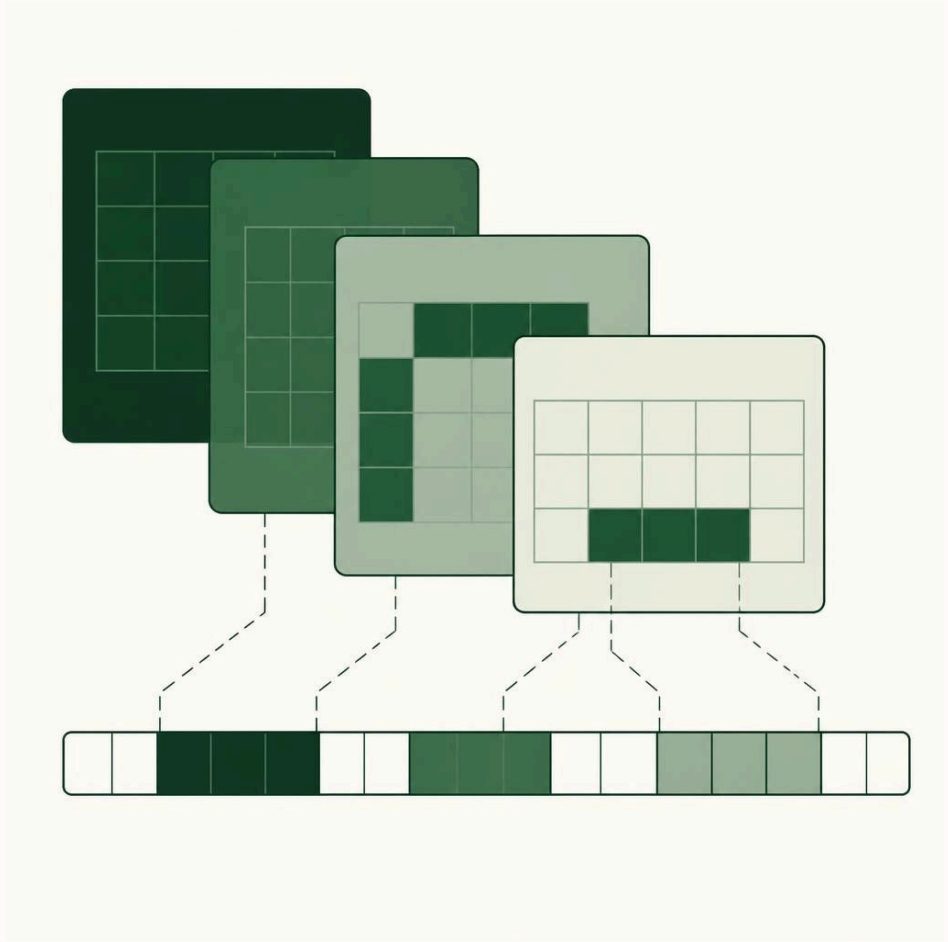
Union in C

Union e memoria, union con etichetta, union in pratica

LINGUAGGIO C | STRUTTURE E UNIONI | UNION

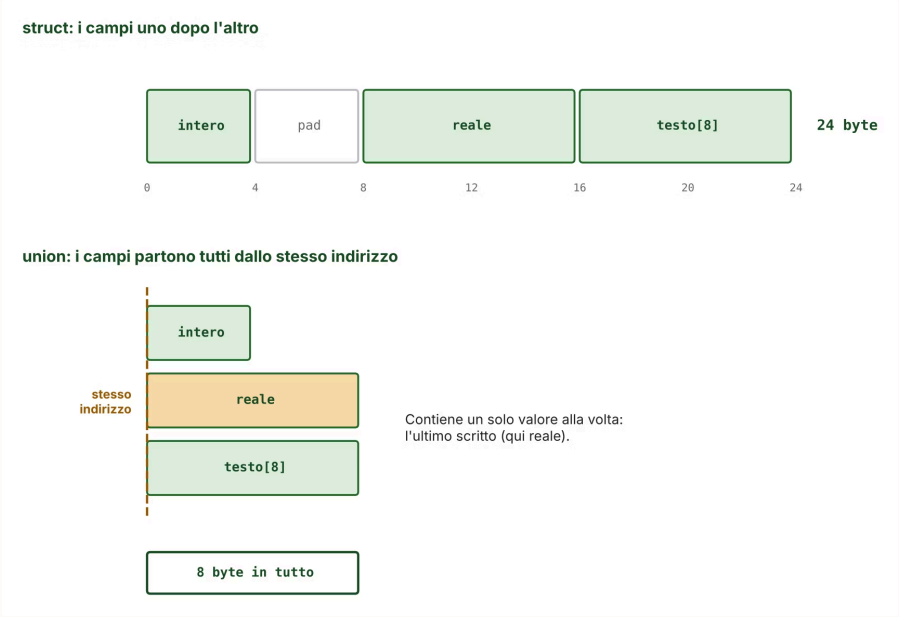


Campi che Condividono la Memoria

Una union si dichiara come una struct, ma i suoi campi cominciano tutti allo stesso indirizzo e si sovrappongono. La dimensione è quella del campo più grande, non la somma dei campi.

```
typedef union {
    int intero;
    double reale;
    char testo[8];
} Valore;

sizeof(TreValori)   /* struct con gli stessi campi: 24 byte */
sizeof(Valore)      /* union: 8 byte */
```



> &v.intero, &v.reale e &v.testo sono lo stesso indirizzo: in ogni momento la union contiene un solo valore.

Un Campo alla Volta

```
v.intero = 65;
v.reale = 3.5;          /* sovrascrive i byte di intero */
printf("%d", v.intero); /* 0: byte del double letti come int */

strcpy(v.testo, "ciao");
printf("%g", v.reale);  /* 3.50000083: valore senza senso */
```

struct	**union**
campi uno dopo l'altro	campi sugli stessi byte
sizeof = somma dei campi	sizeof = campo più grande
tutti i valori insieme	un valore alla volta

> Leggere un campo diverso da quello scritto per ultimo compila, ma reinterpreta i byte presenti: il risultato non ha legame con il valore scritto.

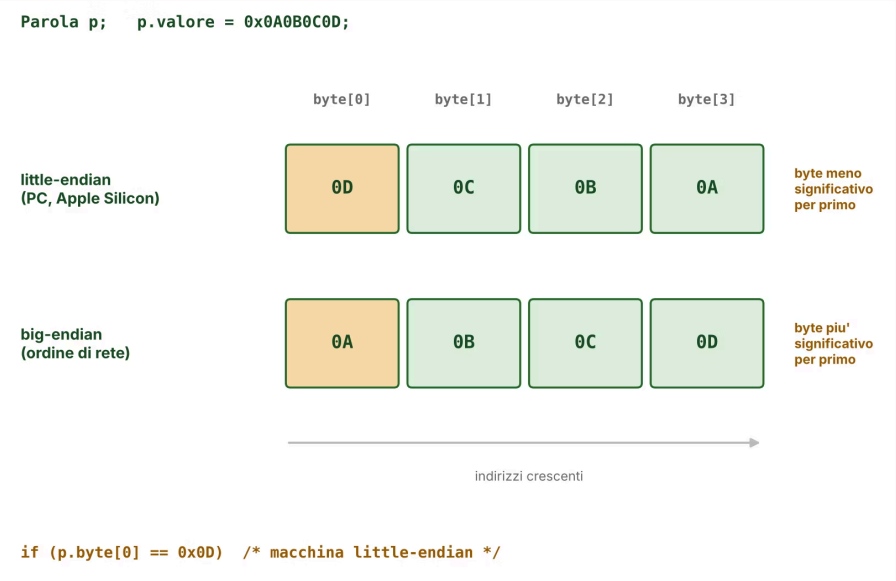
Vedere i Byte di un Numero

Con una union lo stesso valore si guarda in due modi: come numero e come sequenza di byte. I tipi di <stdint.h> hanno una dimensione fissa.

```c
#include <stdint.h>

typedef union {
    uint32_t valore;     /* esattamente 32 bit */
    uint8_t byte[4];     /* esattamente 8 bit ciascuno */
} Parola;

Parola p;
p.valore = 0x0A0B0C0D;
p.byte[0]            /* 0x0D su una macchina little-endian */
```



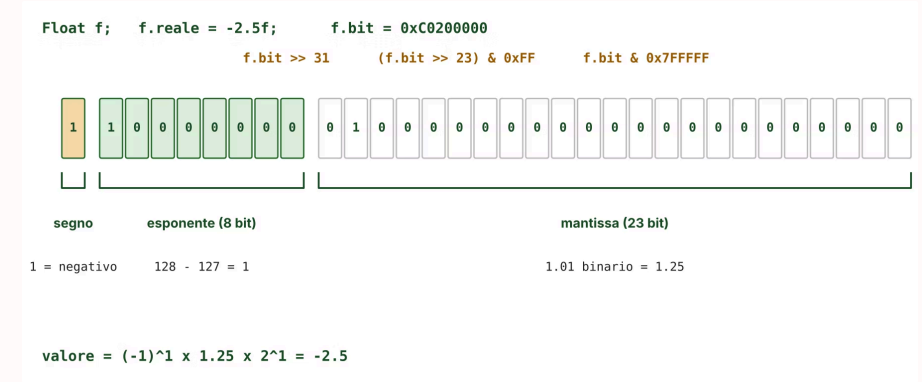
> Little-endian (PC, Mac con Apple Silicon) mette per primo il byte meno significativo; big-endian, l'ordine dei protocolli di rete, il più significativo.

Com'è Fatto un float

Un float occupa 32 bit: segno, esponente e mantissa (IEEE 754). Con una union si leggono i bit come intero senza segno e si separano con spostamenti e maschere.

```
typedef union {
    float reale;
    uint32_t bit;
} Float;

segno     = f.bit >> 31;
esponente = (f.bit >> 23) & 0xFF;     /* meno 127 */
mantissa  = f.bit & 0x7FFFFF;
```



> Lo stesso schema decodifica indirizzi IP, colori RGBA e pacchetti di dati: un array di byte e i campi che quei byte rappresentano.

Risparmiare Memoria

```
typedef struct {
    char codice[8];
    TipoArticolo tipo;
    union {
        Componente componente;   /* 16 byte */
        Strumento strumento;     /* 32 byte */
        Manuale manuale;         /* 28 byte */
    } dati;
} ArticoloCompatto;
```

Un articolo dell'inventario è un componente oppure uno strumento oppure un manuale: le tre alternative si escludono e possono condividere la memoria.

Soluzione	sizeof	500 articoli
struct con tutti i campi	96 byte	48000 byte
struct con union	48 byte	24000 byte

> Il campo tipo dice quale alternativa è valida: è la union con etichetta.

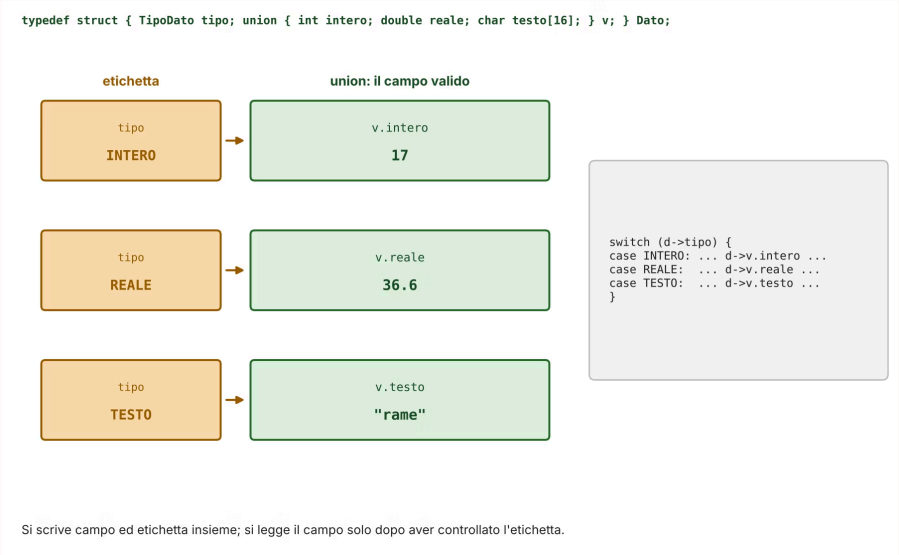
Union con Etichetta

Una union da sola non ricorda quale campo è stato scritto. Si mette in una struct accanto a un campo enum, l'etichetta, che lo dice.

```
typedef enum { INTERO, REALE, TESTO } TipoDato;

typedef struct {
    TipoDato tipo;        /* l'etichetta */
    union {
        int intero;
        double reale;
        char testo[16];
    } v;
} Dato;

d.tipo = REALE;
d.v.reale = 21.5;
```



> Si scrivono campo ed etichetta insieme; si legge un campo solo dopo aver controllato l'etichetta.

Uno switch per Ogni Operazione

```
void stampa(const Dato *d)
{
    switch (d->tipo) {
    case INTERO:
        printf("%d", d->v.intero);
        break;
    case REALE:
        printf("%g", d->v.reale);
        break;
    case TESTO:
        printf("%s", d->v.testo);
        break;
    }
}
```

Ogni operazione ha la stessa forma: uno switch sull'etichetta, con un case per tipo, che usa solo il campo valido.

Due dati sono uguali se hanno la stessa etichetta e, per quel tipo, lo stesso valore: == per i numeri, strcmp per i testi.

> Senza default il compilatore segnala i case mancanti. Confrontare i byte con memcmp non funziona: i byte non usati dal campo valido sono casuali.

Funzioni Costruttrici

```
Dato crea_intero(int valore);
Dato crea_reale(double valore);
Dato crea_testo(const char *valore);

Dato scheda[] = {
    crea_intero(4),
    crea_testo("solfato di rame"),
    crea_reale(21.5)
};
```

Scrivere a mano etichetta e campo in due istruzioni è un invito all'errore. Una funzione per tipo restituisce un Dato già coerente.

Campo	Dove va
comune a tutti i tipi (titolo, importo)	fuori dalla union
specifico di un tipo (raggio, cifre della carta)	dentro la union

> crea_testo copia al massimo 15 caratteri e chiude sempre la stringa: un testo troppo lungo non esce dal campo.

Union Anonima e Accesso Controllato

```
typedef struct {
    TipoDato tipo;
    union {
        int intero;
        double reale;
        char testo[16];
    };                  /* senza nome (C11) */
} Dato;

Dato d = {.tipo = REALE, .reale = 9.81};
d.reale                 /* invece di d.v.reale */

int come_reale(const Dato *d, double *risultato);

if (come_reale(&d, &x)) {
    /* x è valido */
}
```

Dal C11 la union dentro una struct può non avere nome: i suoi campi si usano come campi della struct, ma condividono ancora la memoria.

Una funzione come come_reale controlla l'etichetta e restituisce un esito, con il valore in un parametro di uscita.

> Chi chiama come_reale non tocca mai la union e non può leggere il campo sbagliato.

Errori Comuni da Evitare

1

Leggere un campo della union diverso da quello scritto per ultimo.

2

Pensare che la union conservi tutti i campi insieme.

3

Dare per scontato l'ordine dei byte (little-endian o big-endian).

4

Usare int e char al posto di uint32_t e uint8_t per vedere i byte.

5

Scrivere il campo senza aggiornare l'etichetta.

6

Leggere il campo senza controllare l'etichetta.

7

Mettere default nello switch sull'etichetta e perdere l'avviso.

8

Confrontare due union con memcmp invece che secondo il tipo.

Riepilogo Concetti

Situazione	Tecnica	Note
Alternative che si escludono	union { ... }	sizeof = campo più grande
Un valore alla volta	leggere l'ultimo campo scritto	gli altri sono sovrascritti
Byte di un numero	union { uint32_t; uint8_t[4]; }	dipende dall'endianness
Dimensione fissa	uint8_t, uint32_t, int16_t	da <stdint.h>
Bit di un float	>> e maschere su uint32_t	segno, esponente, mantissa
Ricordare il campo valido	struct con etichetta enum	union con etichetta
Operazioni sul dato	switch sull'etichetta	niente default
Costruire un dato	Dato crea_reale(double)	etichetta sempre coerente
Accesso più corto	union anonima (C11)	d.reale
Lettura sicura	int come_reale(const Dato *, double *)	esito più parametro di uscita

SEZIONE 1

Esempi Pratici: Union e Memoria

Concetti base — tre file su sovrapposizione e byte.

1 – 010-campi-sovrapposti.c

Problema

tre campi in una struct e in una union.

Tecnica

sizeof e indirizzi dei campi.

2 – 020-un-campo-alla-volta.c

Problema

tre scritture successive nella stessa union.

Tecnica

dopo ognuna, solo un campo ha senso.

3 – 030-vedere-i-byte.c

Problema

su che tipo di macchina gira il programma?

Tecnica

uint32_t e uint8_t[4], little-endian.

SEZIONE 1

Esempi Pratici: Problemi su Union e Memoria

040 – Indirizzo IP

Numero e quattro byte: rete, broadcast e stessa rete.

050 – Colore RGBA

Canali indicizzati da un enum: schiarire, scurire, grigio.

060 – Anatomia di un float

Segno, esponente e mantissa di numeri letti da tastiera.

070 – Pacchetto del sensore

Otto byte letti come campi, con il byte di controllo.

080 – Risparmio di memoria

Inventario: struct con tutti i campi contro struct con union.

SEZIONE 2

Esempi Pratici: Union con Etichetta

Concetti base — tre file su etichetta, switch e costruttrici.

1 – 010-etichetta-enum.c

Problema

un valore intero, reale o di testo.

Tecnica

Dato con etichetta, scritti insieme.

2 – 020-switch-sull-etichetta.c

Problema

le risposte di due studenti a un questionario.

Tecnica

stampa e confronto con uno switch.

3 – 030-funzioni-costruttrici.c

Problema

la scheda di un esperimento di chimica.

Tecnica

crea_intero, crea_reale, crea_testo.

SEZIONE 2

Esempi Pratici: Problemi su Union con Etichetta

040 – Figure geometriche

Cerchi, rettangoli e triangoli: area, perimetro e validità.

050 – Celle del foglio

Foglio delle spese della gita con celle vuote, numeri e testi.

060 – Pagamenti al bar

Contanti, carta o buono: ricevuta e totali per metodo.

070 – Catalogo della biblioteca

Libri, DVD e riviste: campi comuni fuori dalla union.

080 – Risultati delle misure

Una misura riuscita, fuori scala o guasta.

SEZIONE 3

Esempi Pratici: Union in Pratica

Concetti base — tre file su union anonima, array e accesso.

1 – 010-union-anonima.c

Problema

la stessa struct con union con nome e anonima.

Tecnica

d.reale invece di d.v.reale.

2 – 020-array-di-valori-misti.c

Problema

una riga delle iscrizioni al torneo di scacchi.

Tecnica

conteggi per tipo, somma dei numeri, testi.

3 – 030-accesso-controllato.c

Problema

i valori letti da un modulo.

Tecnica

come_intero e come_reale con esito.

SEZIONE 3

Esempi Pratici: Problemi su Union in Pratica

040 – Impostazioni del server

File chiave=valore convertito secondo il tipo di ogni chiave.

050 – Comandi del robot

Analisi delle righe del programma ed esecuzione.

060 – Token di un'espressione

Numeri e simboli, calcolo ricorsivo con precedenza.

070 – Registro degli eventi

Porte, sensori e badge: filtro per tipo e anomalie.

080 – Calcolatrice di unità

Un enum di unità per ogni dimensione nella union.